Trabajo Práctico Integrador — API REST con Arquitectura en Capas

Objetivo

Construir un **servidor con Express** que exponga una API REST de un dominio **a tu elección**, aplicando la arquitectura que trabajamos durante las clases:

- Arquitectura en capas con **Inversión de Dependencias** (rutas → controladores → casos de uso → DAO).
- **Persistencia intercambiable** entre memoria y una base de datos real.
- **Composition Root / contenedor** que ensambla las dependencias.
- **Eventos** (`EventEmitter`) para desacoplar reglas de negocio de efectos secundarios.

La idea es que **pongas en práctica lo visto** y lo apliques a un dominio propio, no que copies el ejemplo de la biblioteca.

1. Dominio

Elegí un dominio que tenga **al menos dos entidades relacionadas**. Algunos ejemplos (podés proponer otro):

Dominio	Entidades de ejemplo
Gimnasio	Socios, Clases, Inscripciones
Veterinaria	Dueños, Mascotas, Turnos
Tienda	Clientes, Productos, Pedidos
Cursos	Alumnos, Cursos, Inscripciones
Taller mecánico	Clientes, Vehículos, Reparaciones

No se aceptan la biblioteca ni el CRUD de libros/usuarios de las clases.

2. Requisitos obligatorios

2.1. Servidor y estructura

- Node.js con **ES Modules** y **Express**.
- `index.js` limpio: solo arranca el servidor y monta el router que entrega el contenedor.
- Estructura por capas, con una carpeta por responsabilidad: `routes/`, `controllers/`, `usecases/`, `dao/`, `schemas/`, `middlewares/`, `errors/`, `config/`, `container/`.
- Configuración centralizada en `config/config.js`, leída desde variables de entorno. Incluí un `.env.example` (el `.env` **no** se versiona).

2.2. Entidades y CRUD

- **Mínimo 2 entidades** con CRUD completo (listar, obtener por id, crear, actualizar, eliminar).

- **Al menos una relación** entre ellas (1 a N como mínimo).
- Una de las entidades debe tener **una regla de negocio** que viva en un caso de uso (por ejemplo: "no se puede duplicar el email", "no hay stock disponible", "el turno ya está ocupado").

2.3. Validación y errores

- Validación de entrada con **Zod** (`body`, `params` y `query`), mediante un middleware reutilizable.
- Errores de dominio con una clase propia (`AppError`) con `code`, `message` y `statusCode`.
- **Un único middleware de manejo de errores** y un middleware `notFound`.
- Respuestas de error con un formato consistente (por ejemplo `{ error: { code, message, details } }`).
- Códigos HTTP correctos: 200, 201, 204, 400, 404, 409, 500.

2.4. Persistencia intercambiable

- Cada entidad tiene **dos adaptadores DAO con la misma interfaz**:
  - **En memoria** (array de JavaScript).
  - **Una base de datos real**, a elección:
  - **ORM** (SQL): Sequelize + SQLite, PostgreSQL o MySQL, o Prisma / TypeORM.
  - **ODM** (MongoDB): Mongoose.
- La elección se hace con una variable de entorno (por ejemplo `PERSISTENCE_TYPE`) a través de una **factory** de DAOs.
- Los DAO devuelven **objetos planos** de JavaScript, nunca instancias del ORM/ODM.
- **Cambiar de persistencia no debe requerir tocar** controladores, rutas ni casos de uso.
- Ambos modos deben **responder igual** ante las mismas peticiones (misma forma de respuesta y mismos códigos).

> ***Si elegís ODM (MongoDB):*** *necesitás una instancia (local con Docker o MongoDB Atlas). La URI va en una variable de entorno (`MONGO_URI`). La relación se resuelve con referencias (`ref`) y `populate`, que es el equivalente al `include` de Sequelize.* ***Si elegís ORM (SQL):*** *definí las asociaciones de forma centralizada (como hicimos en `dao/models/index.js`).*

2.5. Inyección de dependencias

- Un **contenedor** (`container/container.js`) que:
  - Resuelve los DAOs con la factory.
  - Los inyecta en los casos de uso (mediante una factory `makeXxxUseCases`).
  - Inyecta los casos de uso en los controladores.
  - Monta los routers y devuelve el router principal.
- Los casos de uso **no importan** DAOs concretos; los reciben por parámetro.
- Los controladores **no conocen** ningún DAO.

2.6. Eventos

- Un **bus de eventos** (`EventEmitter`) creado e inyectado desde el contenedor.
- **Al menos un evento de dominio** emitido por un caso de uso (por ejemplo `pedido.creado`, `stock.bajo`, `turno.cancelado`).
- Un **listener** en un módulo aparte que reaccione al evento mediante un **notificador** con interfaz propia (consola, archivo, etc.), de forma que cambiarlo no requiera tocar la regla de negocio.
- El listener debe **capturar sus propios errores**: un fallo de notificación no debe tumbar la operación.

2.7. Documentación de la API (Swagger / OpenAPI)

- La API debe estar documentada con **OpenAPI 3** y publicada con **Swagger UI** en la ruta `/api-docs`.
- Podés usar `swagger-ui-express` junto con un archivo `openapi.yaml` / `openapi.json`, o `swagger-jsdoc` con comentarios en las rutas. Elegí una y mantenela consistente.
- La documentación debe cubrir **todos los endpoints** e incluir, para cada uno:
  - Método, ruta y descripción breve.
  - Parámetros (`path` y `query`) y **cuerpo** de la petición, con ejemplos.
  - **Respuestas posibles** con su código HTTP y su forma (éxito y errores: 400, 404, 409, etc.).
- Los **schemas** de las entidades y del formato de error deben estar definidos en `components/schemas` y reutilizarse con `$ref`.
- Debe poder **probarse desde Swagger UI** ("Try it out") contra el servidor en ambos modos de persistencia.

2.8. README

Un `README.md` con:

- Descripción del dominio y de las entidades.
- Diagrama o descripción de la relación entre ellas.
- Cómo instalar y ejecutar (incluyendo las variables de entorno).
- Cómo cambiar de persistencia.
- La ruta de la documentación (`/api-docs`).
- Qué evento implementaste y qué hace su listener.

> *El listado de endpoints vive en Swagger: no hace falta duplicarlo en el README.*

3. Requisitos opcionales (suman puntos extra)

- Filtros, ordenamiento y **paginación** en los listados.
- Relación **N a N** con tabla intermedia y una operación que modifique dos entidades de forma **atómica** (transacción).
- **Segundo evento** con un notificador adicional (por ejemplo, archivo y consola).
- **Dockerfile** o `docker-compose.yml` con la base de datos.

- Tests automatizados de los casos de uso (con un DAO en memoria como doble).
- Cambios de seguridad que veamos en próximas clases (por ejemplo hash de contraseñas o JWT).

4. Restricciones

- El trabajo es **preferentemente grupal**, aunque también puede entregarse de forma **individual**.
- En el trabajo grupal, **todos los integrantes** deben aportar commits propios y responder en la defensa sobre cualquier parte del proyecto.
- No subir `node_modules`, `.env` ni archivos de base de datos (`.sqlite`).
- Se pueden usar herramientas de IA para consultar, pero **tenés que poder explicar y modificar cada parte de tu código** en la defensa. Una API que funciona pero que no sabés explicar no aprueba.

5. Entregables

1. Repositorio de Git con **commits progresivos** (no un solo commit final).
2. Código fuente según lo anterior.
3. `README.md`, `.env.example` y la documentación Swagger en `/api-docs`.
4. **Defensa oral** breve: se te pedirá cambiar de persistencia en vivo y agregar o modificar una regla pequeña.

6. Criterios de evaluación

Criterio	Puntos (sobre 10)
Arquitectura en capas y separación de responsabilidades	2
Persistencia intercambiable (dos DAO con la misma interfaz y factory)	2
Contenedor e inyección de dependencias	0,5
Validación (Zod) y manejo de errores centralizado	1,5
Reglas de negocio en casos de uso y relación entre entidades	1
Eventos: emisor, listener y notificador desacoplados	1
Documentación (Swagger completa y README)	1
Calidad del código (nombres, consistencia, commits)	0,5
Defensa oral	0,5
Total	**10**

Los requisitos opcionales suman hasta **1 punto extra** (sobre la nota de 10).

7. Sugerencia de planificación

1. **Definir el dominio**: entidades, campos y relaciones (en papel).

2. **Primero en memoria**: schemas, DAO en memoria, casos de uso, controladores, rutas y contenedor. Probá todo desde Swagger UI.
3. **Agregar la persistencia real**: segundo DAO con la misma interfaz y la factory. Verificá que ambos modos respondan igual.
4. **Relaciones y reglas de negocio**.
5. **Eventos y notificador**.
6. **Documentación Swagger, README, pruebas finales y limpieza**.

> *Trabajar primero en memoria permite validar toda la arquitectura sin depender de la base de datos. Si la capa HTTP y el dominio están bien separados, sumar la persistencia real es solo agregar un adaptador.*

¡Éxitos!